Quiz — 2.1.1 Thinking Abstractly

Name: ____ Date: _ ***Mark:*** ___ / 20

OCR H446 · Component 02 · 2.1.1

Section A — Multiple choice (6 marks)

1. A team modelling a city's road network for a route-finding app represent junctions as nodes and roads as weighted edges, ignoring road surface and street names. This is an example of: A. Decomposition B. Representational abstraction C. Caching D. A precondition Your answer: ☐

2. A developer designing a banking app creates one `Account` class used for both savings and current accounts by capturing their shared characteristics. Which form of abstraction is this? A. Representational abstraction B. Procedural ordering C. Generalisation / categorisation D. Concurrency Your answer: ☐

3. For a chess-playing program, which detail is it most appropriate to **discard** through abstraction? A. The rules governing how each piece moves B. The current position of every piece C. The material and weight of the physical chess pieces D. Whose turn it is to move Your answer: ☐

4. A student claims "abstraction means breaking the problem into smaller sub-problems." This is wrong because that describes: A. Caching B. Decomposition C. Generalisation D. A decision point Your answer: ☐

5. The London Underground map shows lines and stations but not the true geographic distance or curve of the track. The map is useful because abstraction has: A. Added extra detail to help navigation B. Removed detail irrelevant to the task of choosing a route C. Split the journey into sub-procedures D. Run several tasks at once Your answer: ☐

6. A weather simulation models the atmosphere as a 3-D grid of cells. Choosing to represent continuous air as discrete cells is a deliberate act of: A. Validation B. Abstraction (building a simplified model of reality) C. Deadlock D. Reusable components Your answer: ☐

Section B — Short answer (9 marks)

7. A games studio is building a top-down racing game. Identify **two** real-world details they would *keep* in their model of a car and **one** detail they would *discard*, justifying why the discarded detail is irrelevant to this problem. **[3]**

8. Explain the difference between *representational abstraction* and *generalisation*, using a clear example of each. **[2]**

9. A satnav represents a country's road system as a graph of nodes and weighted edges. Explain what the nodes and the edge weights represent, and why this abstraction is appropriate for finding the fastest

route. **[2]**

10. State why a generalised, abstract model (e.g. a single `Shape` type for circles and squares) can make a program easier to extend later. **[2]**

Section C — Applied question (5 marks)

11. A hospital wants software to manage patient appointments. Explain how the developers would use abstraction to design a model of a "patient" for this system. Your answer should name specific data the model would **keep**, specific real-world detail it would **discard**, and justify *why* each discarded detail is irrelevant to the appointments problem. **[5]**

Answer key

Section A (6 marks — 1 each) 1. **B** — modelling one specific thing (the road network) as a simpler representation (a graph) is representational abstraction. 2. **C** — grouping savings and current accounts by their shared characteristics into one general type is generalisation/categorisation. 3. **C** — the physical material/weight has no bearing on play; the game only needs piece positions, legal moves and whose turn it is. 4. **B** — breaking into sub-problems is decomposition; abstraction *removes/hides* detail. Common trap. 5. **B** — discarding true geography leaves only what matters for choosing a route (lines, order of stations, interchanges). 6. **B** — representing continuous reality as a simplified discrete grid is abstraction (building a workable model).

Section B (9 marks)

7. **[3]** Keep (any two, 1 each): position/coordinates on track; speed/velocity; direction/heading; fuel or tyre state if it affects play. Discard + justify (1): e.g. engine chemistry / exact paint colour / VIN — these do not affect how the car behaves or is driven in the race, so they are irrelevant to the simulation. Must justify the discard for the mark.

8. **[2]** Representational abstraction = simplifying *one specific* thing into a model (1) — e.g. a tube map of one city's network. Generalisation = grouping *many* things by shared characteristics into a reusable model (1) — e.g. one `Vehicle` class for cars and lorries. Award only with a valid example for each idea.

9. **[2]** Nodes represent junctions/places and edge weights represent the cost of travelling between them (distance or time) (1); this is appropriate because a shortest-path algorithm can then work purely on the weights, ignoring irrelevant detail like scenery or surface, to find the fastest route (1).

10. **[2]** A general model captures behaviour common to all cases (1), so a new case (e.g. a triangle) can be added by reusing/extending the existing model rather than rewriting code, reducing effort and bugs (1).

Section C (5 marks) 11. **[5]** Up to 5 from: Abstraction means building a simplified model that keeps only detail relevant to managing appointments (1). Keep — patient ID/name, date of birth, contact number, GP/clinic, required appointment type/duration (any, 1). Discard — e.g. eye colour, favourite food, home décor (1). Justification — such detail plays no part in scheduling or contacting the patient, so including it would add complexity without value (1). Reasoned link that removing irrelevant detail makes the system simpler, faster to build and easier to maintain while still solving the appointments problem (1). Must apply to *this* scenario, not give a generic definition.

Total: 6 + 9 + 5 = 20